

JAVA PROGRAMMING PROJECT
Object-Oriented Application Development

INVENTORY TRACKING SYSTEM

Project Analysis & Design Document

Technology: NetBeans IDE | Language: Java

2025 – 2026 Academic Year

Table of Contents

1. Project Overview
2. Object Descriptions
3. Special Rules & Complex Operations
4. UML Class Diagram
5. Screen Descriptions
6. Mathematical Operations
7. OOP Features Used
8. Exception Handling
9. Technology Stack

1. Project Overview

The Inventory Tracking System is a desktop application developed using Java and NetBeans IDE. It allows users to manage products (Electronic and Food), track stock movements (stock-in / stock-out), monitor critical stock levels, and perform basic inventory analytics. The system enforces user authentication (login / register) and maintains a full log of all stock transactions.

The application consists of four screens: Login Screen, Main Screen, Product Management Screen, and Stock Movements Screen. All business logic resides in the object layer (InventoryManager) and the UI layer only calls these methods.

2. Object Descriptions

Product	Base class representing a product in the inventory. Holds common fields: id, barcode, name, category, quantity, price, criticalLevel. Provides isBelowCritical(), getType(), getExtraInfo() and toString() methods. Acts as the superclass for ElectronicProduct and FoodProduct.
ElectronicProduct	Extends Product. Adds brand (String) and warrantyMonths (int) fields specific to electronic goods. Overrides getType() to return 'Elektronik' and getExtraInfo() to include brand and warranty information.
FoodProduct	Extends Product. Adds expiryDate (Date) and supplier (String) fields. Overrides getType() and getExtraInfo(). Provides isExpired() which checks whether the expiry date has passed relative to the current date.
InventoryManager	Central manager class. Stores all products, categories, stock movements and users in ArrayLists. Provides all business-logic methods: addProduct, removeProduct, stockIn, stockOut, login, register, getTotalValue, getAveragePrice, getTotalStock, getCriticalProducts, getByCategory.
InventoryManager.User (inner)	Inner static class of InventoryManager. Stores username, password and fullName. Provides checkPassword() for authentication. Instances are created via register() and checked via login().
InventoryManager.StockMovement (inner)	Inner static class of InventoryManager. Records each stock-in or stock-out event with productName, barcode, type (IN/EXIT), amount, note and an auto-generated date string. Used to populate the movements table in StockScreen.

3. Special Rules & Complex Operations

Rule 1 — Duplicate Barcode Prevention

Method: InventoryManager.addProduct(Product p)

When a new product is added via addProduct(), the system iterates through all existing products and compares barcodes (case-insensitive). If a match is found, an Exception is thrown and the product is not added. This ensures every product has a unique identifier in the system.

Rule 2 — Stock Cannot Go Below Zero

Method: InventoryManager.stockOut(int id, int amount, String note)

When `stockOut()` is called, the system verifies that the requested amount does not exceed the current quantity. If `quantity < amount`, an `Exception` is thrown with a descriptive message showing both available and requested amounts. This prevents negative stock values.

Rule 3 — Critical Stock Level Warning

Method: `Product.isBelowCritical() + StockScreen.doMovement()`

After every stock movement (in or out), the system checks `isBelowCritical()` on the affected product. If `quantity <= criticalLevel`, a warning dialog is shown in the UI and the product is highlighted in the critical products counter on `MainScreen`. This rule spans both the object layer and the UI layer.

Rule 4 — Food Product Expiry Check

Method: `FoodProduct.isExpired()`

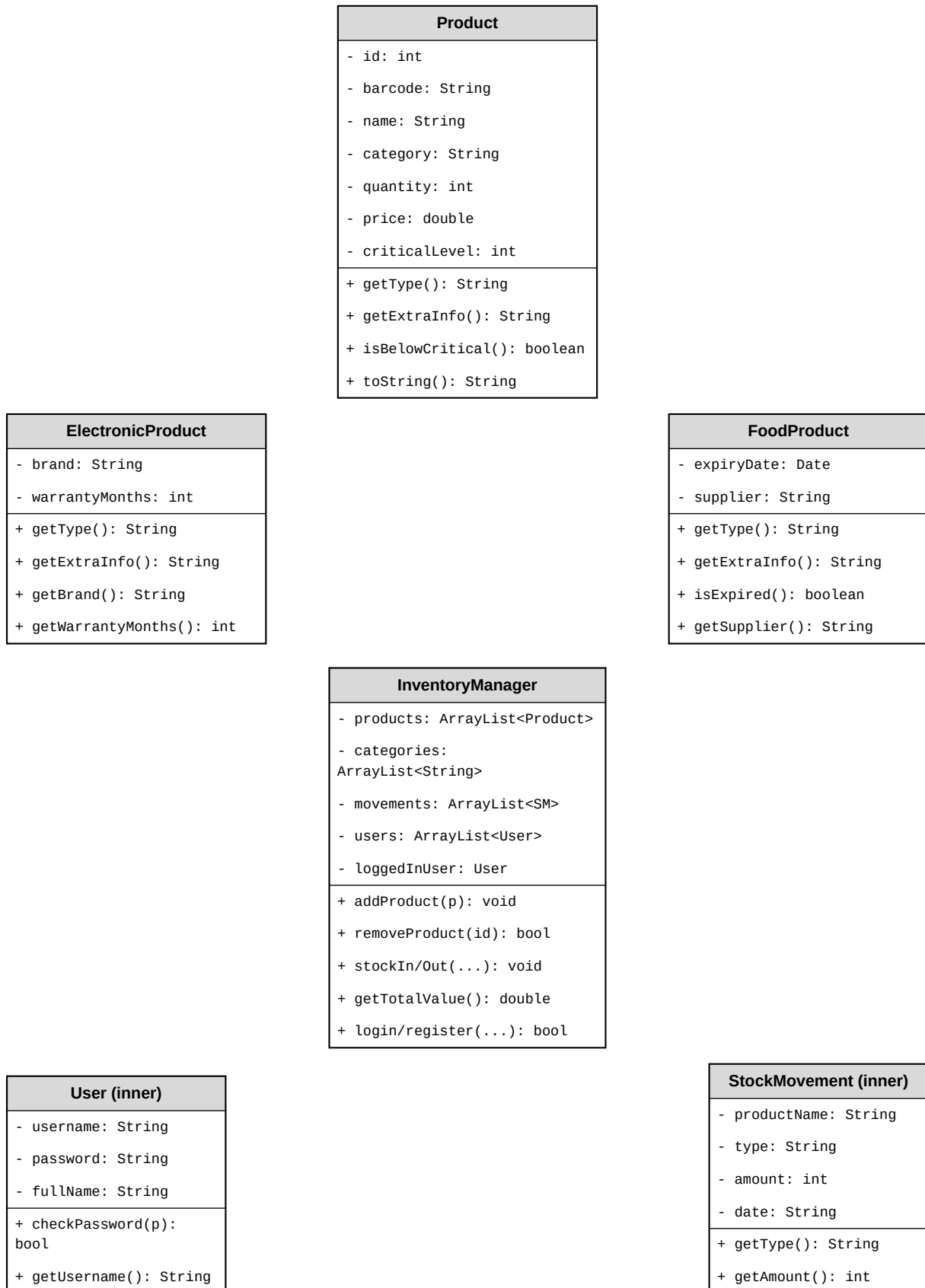
`FoodProduct` provides `isExpired()` which compares `expiryDate` to the current system date. This result is shown in `getExtraInfo()` (appended as 'SÜRESİ DOLDU') and is visible in the product table on `MainScreen`. Products past their expiry date are visually distinguishable.

Rule 5 — Category Cascade on Deletion

Method: `InventoryManager.removeCategory(String name)`

When a category is removed via `removeCategory()`, all products belonging to that category are automatically reassigned to the default 'Genel' category before the category entry is deleted. The 'Genel' category itself is protected and cannot be removed.

4. UML Class Diagram



LoginScreen	MainScreen	ProductScreen	StockScreen
- manager: InventoryManager	- manager: InventoryManager	- manager: InventoryManager	- manager: InventoryManager
+ refreshScreen()	+ refreshScreen()	+ refreshScreen()	+ refreshScreen()
+ initComponents()	+ initComponents()	+ initComponents()	+ initComponents()

Legend: Hollow-triangle arrows = Inheritance | Filled-triangle arrows = Association | Dashed lines = Inner-class / Usage

Figure 1 – UML Class Diagram of the Inventory Tracking System.

5. Screen Descriptions

LoginScreen	The entry point of the application. Contains username and password fields, Login and Register buttons, and a status label for feedback. On successful login, opens mainScreen passing the shared InventoryManager instance.
MainScreen	The dashboard screen. Displays all products in a JTable with columns: ID, Barcode, Name, Category, Type, Stock, Price, Extra Info. A JComboBox allows category-based filtering. Summary labels show Total Value, Average Price, Total Stock, and Critical Product count. Navigation buttons lead to ProductScreen and StockScreen.
ProductScreen	The product management screen. A JList on the left shows all products. Selecting an item fills the form fields. The form includes: Type (JComboBox), Barcode, Name, Category (JComboBox), Quantity (JSpinner), Price, Critical Level, and two dynamic extra fields (Brand + Warranty for Electronics; Supplier + ExpiryDate for Food). Buttons: Add, Update, Delete, Back.
StockScreen	The stock movements screen. A JComboBox lists all products with their current stock. JSpinner selects amount. A note field captures a reason. Stock Entry and Stock Exit buttons trigger stockIn()/stockOut(). A warning label turns red when stock is critical. A JTable shows the full movement history.

6. Mathematical Operations

Total Inventory Value	Iterates all products and sums price × quantity for each. Displayed on MainScreen as 'Total Value'. Method: InventoryManager.getTotalValue()
Average Product Price	Sums all product prices and divides by the product count. Returns 0 if no products exist. Displayed on MainScreen as 'Avg. Price'. Method: InventoryManager.getAveragePrice()
Total Stock Count	Sums the quantity field of all products. Displayed on MainScreen as 'Total Stock'. Method: InventoryManager.getTotalStock()

7. Object-Oriented Programming Features

Inheritance	ElectronicProduct and FoodProduct both extend the Product base class, inheriting all common fields and methods while adding their own specialised attributes.
Polymorphism	getType() and getExtraInfo() are declared in Product and overridden in each subclass. The UI calls these methods on Product references, allowing uniform display of all product types in the JTable.
Encapsulation	All fields in every class are private. Access is exclusively through getter and setter methods, ensuring controlled modification.
ArrayList	Four ArrayLists are maintained in InventoryManager: products, categories, movements, and users. These replace raw arrays for dynamic sizing.
Inner Classes	User and StockMovement are static inner classes of InventoryManager, logically grouping related data structures within the manager.

Exception Handling Custom Exception objects are thrown by `addProduct()` and `stockOut()` for business-rule violations, and caught in the UI layer to display user-friendly error dialogs.

8. Exception Handling

The following exception types are handled throughout the application:

Exception (custom) Thrown by `addProduct()` for duplicate barcodes and by `stockOut()` for insufficient stock. Caught in `ProductScreen.btnAddActionPerformed()` and `StockScreen.doMovement()` respectively.

NumberFormatException Caught in `ProductScreen` when price, critical level, or warranty months fields contain non-numeric input. Displays an appropriate error dialog.

ParseException Caught in `ProductScreen` when the expiry date field for `FoodProduct` does not match the expected `dd/MM/yyyy` format.

General Exception A catch-all Exception block in both `ProductScreen` and `StockScreen` prevents uncaught runtime errors from crashing the application.

9. Technology Stack

Language	Java (JDK 17+)
IDE	Apache NetBeans IDE 28
UI Framework	Java Swing (JFrame, JTable, JList, JComboBox, JSpinner, JOptionPane)
Build Tool	Apache Maven (managed by NetBeans)
Data Storage	In-memory ArrayLists (no external database)
Logging	<code>java.util.logging.Logger</code> used in all UI classes